Ejecucion

mvn clean spring-boot:run

primero:

mvn clean install

Ejecutarlo usando Maven Wrapper (RECOMENDADO)

Como tu proyecto tiene `.mvn`, puedes usar el wrapper sin instalar Maven.

Desde la raíz del proyecto (`COMPUNET2-202601`) ejecuta:

En Windows:

mvnw.cmd spring-boot:run

O:

.\mvnw.cmd spring-boot:run

Modelo para model:

```
@Entity
@Table(name = "survey")
public class Survey {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;
}
```

BaseDeDatos

http://localhost:8081/compunet2-2026/h2-console

```
# Is DB_CLOSE_DELAY=-1 necessary? What does it do? It is useful to keep the database open after the last
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:mem:db-kevin;DB_CLOSE_DELAY=-1
spring.datasource.username=postgres
spring.datasource.password=postgres
# None, create, update, create-drop
spring.jpa.hibernate.ddl-auto=create-drop
# Run data.sql or schema.sql on startup
spring.sql.init.mode=always
spring.jpa.show-sql=false
# This property is used to defer the datasource initialization until after the application context is re
spring.jpa.defer-datasource-initialization=true
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

🧠 Cómo saber con certeza cuál base estás usando

Abre tu `application.properties` y revisa:

🔵 Si ves esto:

```properties
spring.datasource.url=jdbc:h2:mem:db-kevin
```

👉 Estás usando H2
👉 PostgreSQL no participa en nada

🟢 Si vieras algo como esto:

```properties
spring.datasource.url=jdbc:postgresql://localhost:5432/compunet2
spring.datasource.username=postgres
spring.datasource.password=1234
```

👉 Entonces sí estarías usando PostgreSQL
👉 Y deberías ver las tablas en pgAdmin

---

lo que define como se debe llamar la base de datos en postgres:

👉 El nombre `compunet2` NO está en el código Java.
👉 Está en el `application.properties`.

🎯 ¿Dónde se define el nombre de la base?

En esta línea:

```properties
spring.datasource.url=jdbc:postgresql://localhost:5432/compunet2
```

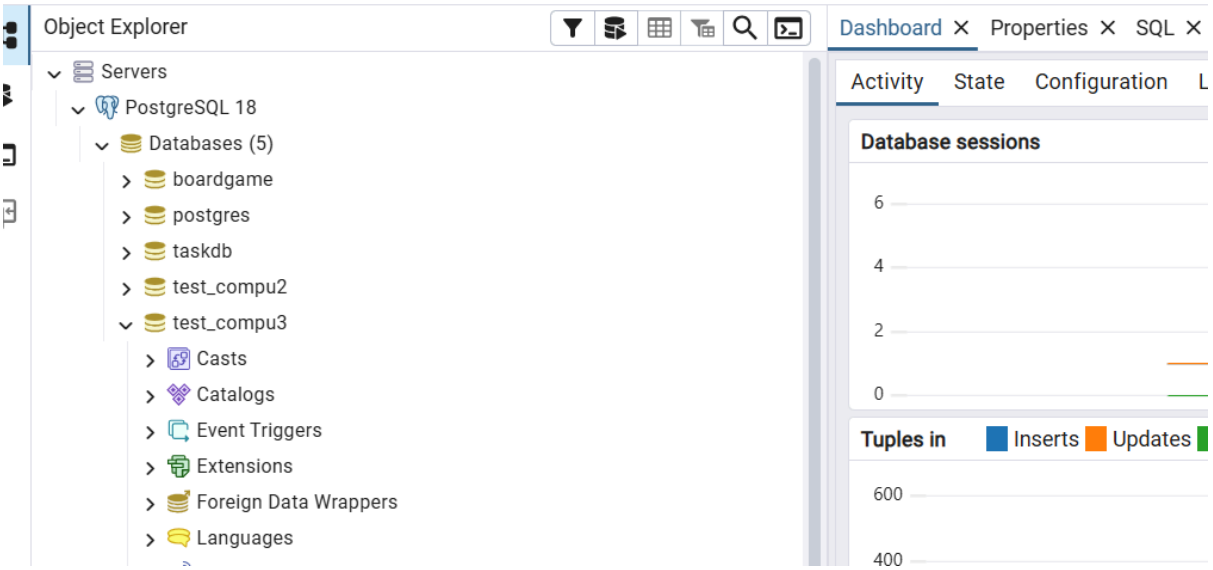
La parte final:

```
/compunet2
```

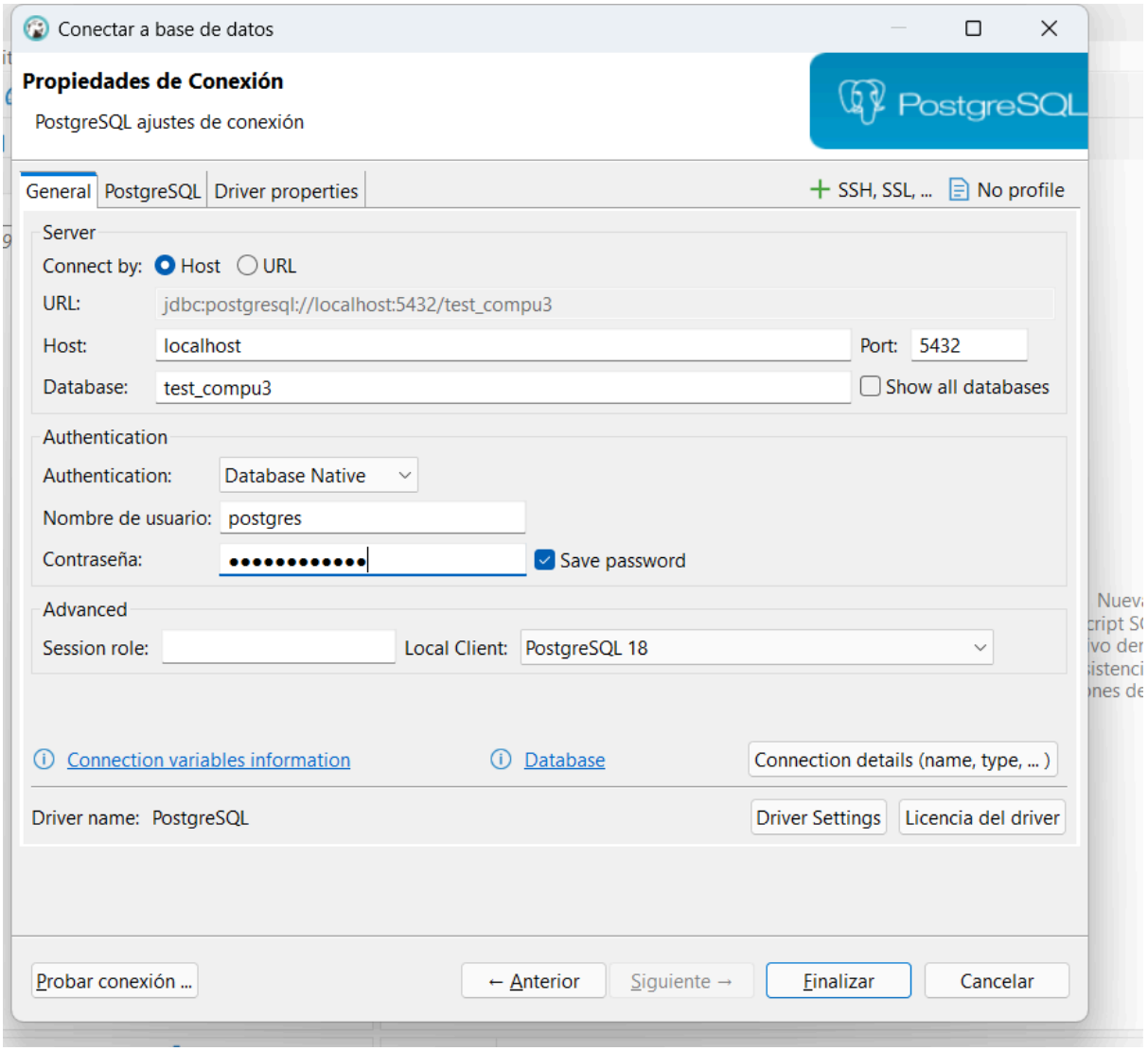
- Ese es el nombre de la base de datos en PostgreSQL.

---

es



Este nombre de base de datos es el que se coloca en Dbeaver


Conectar a base de datos
Propiedades de Conexión
PostgreSQL ajustes de conexión
PostgreSQL
General
PostgreSQL
Driver properties
SSH, SSL, ...
No profile
Server
Connect by:
Host
URL
URL:
jdbc:postgresql://localhost:5432/test_compu3
Host:
localhost
Port:
5432
Database:
test_compu3
Show all databases
Authentication
Authentication:
Database Native
Nombre de usuario:
postgres
Contraseña:
Save password
Advanced
Session role:
Local Client:
PostgreSQL 18
Connection variables information
Database
Connection details (name, type, ...)
Driver name: PostgreSQL
Driver Settings
Licencia del driver
Probar conexión ...
← Anterior
Siguiente →
Finalizar
Cancelar

interpretacion_relaciones

La forma de leer cada línea es **siempre igual**:

Ref: tablaA.campo_fk > tablaB.campo_pk

significa:

- `tablaA.campo_fk` **es una llave foránea**
- apunta a `tablaB.campo_pk`
- por lo tanto:

Muchos registros de tablaA → pertenecen a uno de tablaB

o sea:

tablaA = ManyToOne
tablaB = OneToMany

Ahora interpretemos **cada relación exactamente**.

1

Ref: section.survey_id > survey.id

Interpretación:

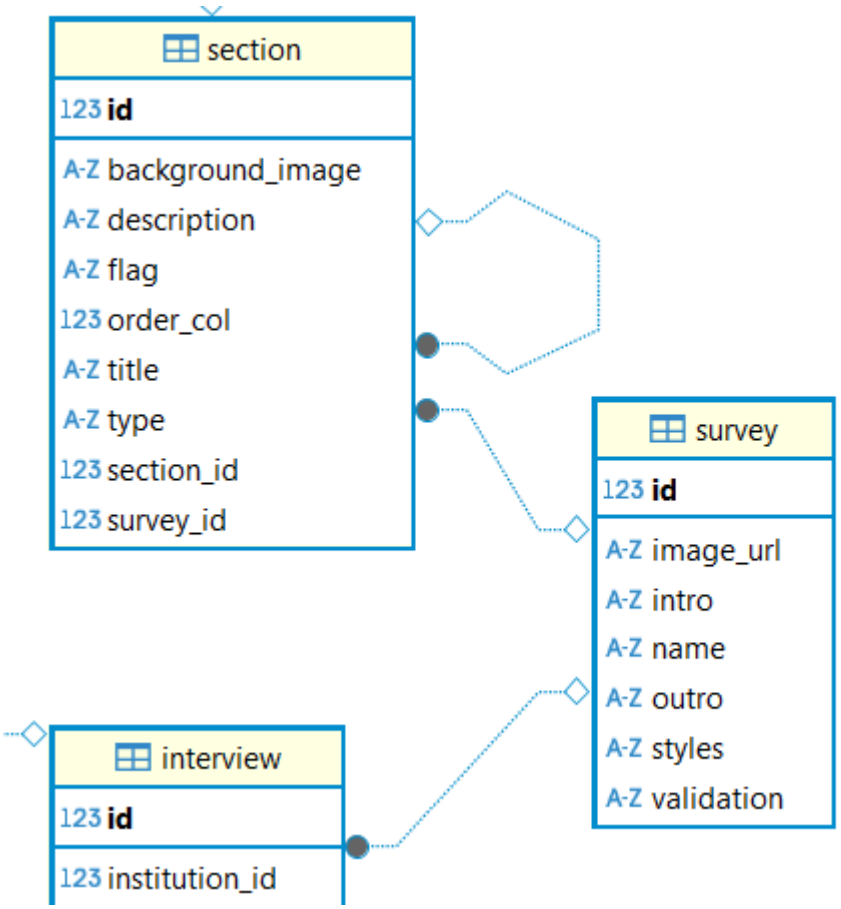
- `section.survey_id` es una FK
- apunta a `survey.id`

Significa:

Muchas sections pertenecen a una survey

Relación:

Survey (1) ------ (N) Section



El uno es la cosa vacia, el circulo rellenado es muchos.

2

Ref: section.section_id > section.id

Interpretación:

- una sección puede apuntar a **otra sección**

Significa:

una sección puede tener una sección padre

Relación:

Section (1) ----- (N) Section

Es una **auto-relación jerárquica**.

Ejemplo conceptual:

```
Sección A
  ├── Sección B
  └── Sección C
```

3

Ref: question.section_id > section.id

Interpretación:

Muchas preguntas pertenecen a una sección

Relación:

Section (1) ------ (N) Question

4

Ref: question.question_id > question.id

Interpretación:

una pregunta puede tener una pregunta padre

Relación:

Question (1) ---- (N) Question

Es otra **auto-relación**.

Puede representar:

```
Pregunta principal
  ├── Subpregunta 1
  └── Subpregunta 2
```

5

Ref: option.group_id > option_group.id

Interpretación:

Muchas opciones pertenecen a un grupo de opciones

Relación:

OptionGroup (1) ------ (N) Option

Ejemplo:

```
Grupo: Colores
  ├ rojo
  ├ azul
  └ verde
```

6

Ref: option_question.question_id > question.id

Interpretación:

Muchos registros de option_question pertenecen a una pregunta

Relación:

Question (1) ------ (N) OptionQuestion

7

Ref: option_question.option_id > option.id

Interpretación:

Muchos registros de option_question pertenecen a una opción

Relación:

Option (1) ------ (N) OptionQuestion

8

Ref: option_question.group_id > option_group.id

Interpretación:

Muchos registros de option_question pertenecen a un grupo

Relación:

OptionGroup (1) ------ (N) OptionQuestion

9

Ref: answer.question_id > question.id

Interpretación:

Muchas respuestas pertenecen a una pregunta

Relación:

Question (1) ------ (N) Answer

10

Ref: answer.interview_id > interview.id

Interpretación:

Muchas respuestas pertenecen a una entrevista

Relación:

Interview (1) ------ (N) Answer

11

Ref: interview.survey_id > survey.id

Interpretación:

Muchas entrevistas pertenecen a una encuesta

Relación:

Survey (1) ------ (N) Interview

12

Ref: question.type > type.name

Interpretación:

Muchas preguntas pertenecen a un tipo

Relación conceptual:

Type (1) ------ (N) Question

Pero aquí **la referencia es a `type.name` en vez de `id`**.

Resumen total de relaciones

Tabla A	Tabla B	Relación
Section	Survey	N → 1
Section	Section	N → 1
Question	Section	N → 1
Question	Question	N → 1
Option	OptionGroup	N → 1
OptionQuestion	Question	N → 1
OptionQuestion	Option	N → 1
OptionQuestion	OptionGroup	N → 1
Answer	Question	N → 1
Answer	Interview	N → 1
Interview	Survey	N → 1
Question	Type	N → 1

💡 Regla simple que usa tu profesor:

Si ves:

tablaA.algo_id > tablaB.id

automáticamente significa:

Muchos A pertenecen a un B

o sea:

A = ManyToOne
B = OneToMany

repository

Hazte esta pregunta:

¿Voy a consultar, guardar, actualizar o borrar esta entidad?

Si la respuesta es **sí**, entonces **debe tener Repository**.

Json

Si no se ingresa el @Data la información no aparece, solo [{},{}] por ejemplo.

Si no se ingresas:

```
    @ToString.Exclude
    @JsonIgnore
```


```
[
  {
    "answer": "Juan",
    "id": 1,
    "interview": {

    },
    "question": {

    }
  },
  {
    "answer": "25",
    "id": 2,
    "interview": {

    },
    "question": {

    }
  },
  {
    "answer": "Excelente",
    "id": 3,
    "interview": {

    },
    "question": {

    }
  }
]
```


si se ingresa (tomar en cuenta que la interview es la misma(http://localhost:8081/compunet2-2026/answer/interview/1)):


```
[
  {
    "answer": "Juan",
    "id": 1
  },
  {
    "answer": "25",
    "id": 2
  },
  {
    "answer": "Excelente",
    "id": 3
  }
]
```

Con Json:

```
30
31      @ManyToOne
32      @ToString.Exclude
33      @JsonIgnore
34      @JoinColumn(name = "interview_id")
35      private Interview interview;
36
37  }
```

localhost:8081/compunet2-2026/answer/interview/1

html cine musi libros Esta fisc e inge bot ja

Dar formato al texto


```
[
  {
    "answer": "Juan",
    "id": 1
  }
]
```


Sin Json

localhost:8081/compunet2-2026/answer/interview/1

html cine musi libros Esta fisc e inge bot java conta

Dar formato al texto


```
[
  {
    "answer": "Juan",
    "id": 1,
    "interview": {
      "id": 1,
      "institutionId": 1,
      "interviewerId": 1,
      "survey": {
        "id": 1,
        "imageUrl": "https://example.com/images/clientes.jpg",
        "intro": "Bienvenido a nuestra encuesta de clientes",
        "name": "Encuesta Clientes",
        "outro": "Gracias por participar",
        "styles": "default",
        "validation": "none"
      },
      "timeEnd": "2026-03-01T10:10:00",
      "timeStart": "2026-03-01T10:00:00",
      "username": "juan"
    }
  }
]
```

LIKE

Cuando el ejercicio dice algo como:

> **“Buscar encuestas por nombre (like)”**

o

> **“Buscar por similitud”**

se refiere a usar `LIKE` **en SQL**, es decir, **buscar coincidencias parciales**.

🔎 Qué significa “por similitud”

En SQL sería algo así:

```
SELECT *
FROM survey
WHERE name LIKE '%cliente%';
```

Esto encuentra:

name
Encuesta Clientes
Encuesta Clientes VIP
Clientes Satisfechos

Porque **contienen la palabra "cliente"**.

En Spring Data JPA

Eso se traduce a:

findByNameContaining(String name)

o mejor:

findByNameContainingIgnoreCase(String name)

Ejemplo:

```
surveyRepository.findByNameContainingIgnoreCase("cliente");
```

🔗 Ejemplo completo

Repository

```
List<Survey> findByNameContainingIgnoreCase(String name);
```

Service

```
public List<Survey> searchByName(String name){
  return surveyRepository.findByNameContainingIgnoreCase(name);
}
```

Controller

```
@GetMapping("/search/{name}")
public List<Survey> search(@PathVariable String name){
  return surveyService.searchByName(name);
}
```

🌐 URL para probar

http://localhost:8081/survey/search/clientes

Consulta_x_URL

Aqui las consultas se hacen por url:

1️⃣ La URL es la que filtra la pregunta

Tu endpoint es:

@GetMapping("/question/{id}/option-question")

Entonces **el `{id}` de la URL decide qué pregunta se consulta**.

Ejemplo:

/option/question/1/option-question

significa:

traer las opciones de la pregunta 1

```sql
-- ========================
-- OPTIONS (AHORA SIN ERRORES DE CODIFICACIÓN)
-- ========================
INSERT INTO option (id, name, type, group_id) VALUES
(1, 'Excelente', 'choice', 1),
(2, 'Bueno', 'choice', 1),
(3, 'Regular', 'choice', 1),
(4, 'Sí', 'choice', 2),
(5, 'No', 'choice', 2),
(6, 'Juan Pérez', 'choice', 1),
(7, 'María López', 'choice', 2),
(8, 'Carlos Ramírez', 'choice', 1),
(9, '18-25 años', 'choice', 2),
(10, '26-35 años', 'choice', 1),
(11, '36 o más', 'choice', 2);

-- ========================
```

Qué hace la tabla `option_question`

La tabla:

option_question

es la que **conecta preguntas con opciones**.

Cada fila significa:

esta opción pertenece a esta pregunta

Pestaña 8

1️⃣ `JpaRepository` ya trae los métodos básicos

Cuando extiendes:

JpaRepository<OptionGroup, Integer>

heredas automáticamente métodos como:

findById()
findAll()
save()
deleteById()
existsById()

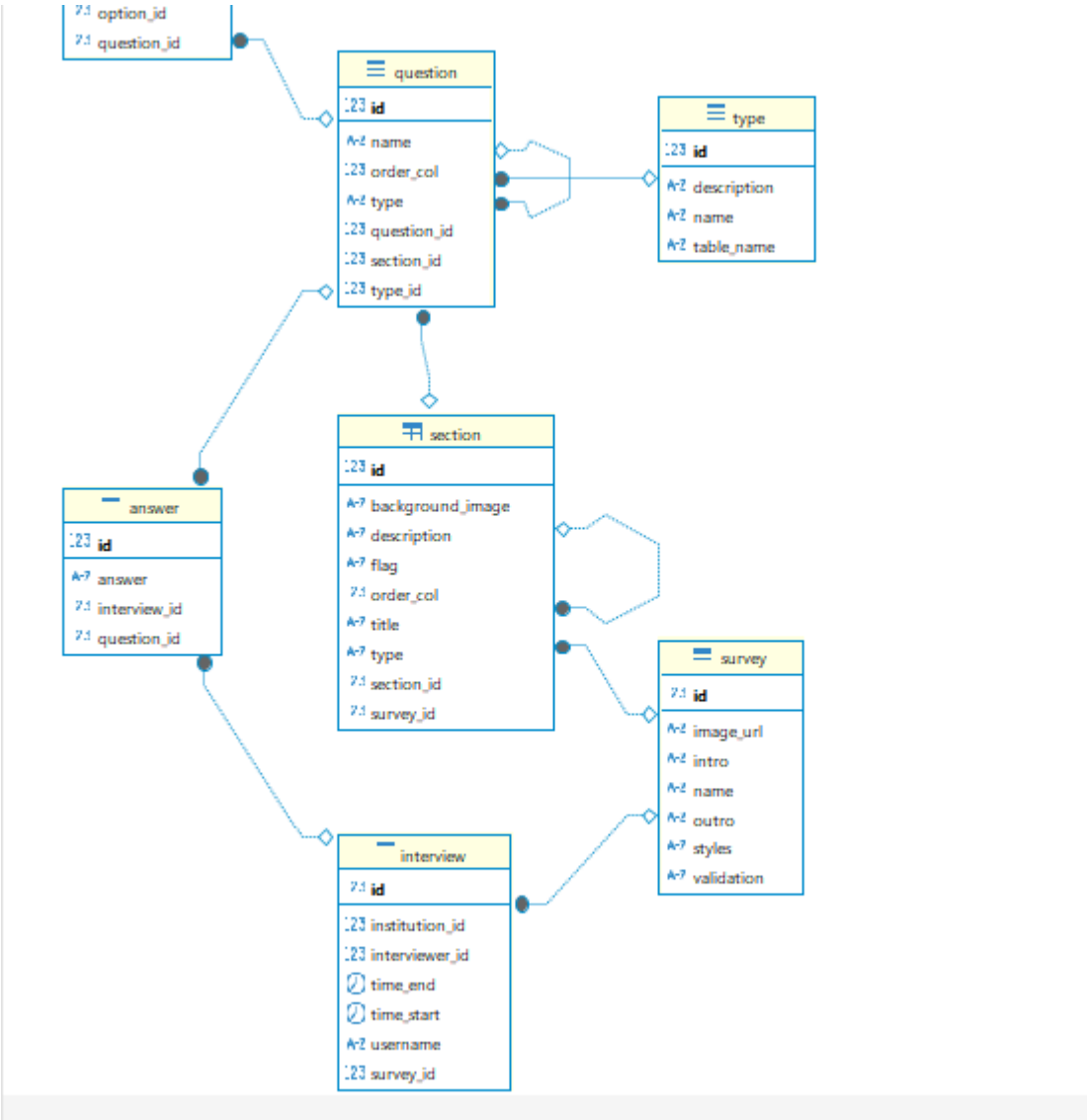
Para la consulta 10 **solo necesitas** `findById()`.

por eso para la consulta 10 no es necesario nada ahi:

```
6
7   @Repository
8   public interface IOptionGroupRepository extends JpaRepository<OptionGroup, Integer> {
9       // Esto basta para la pregunta 10.
10  }
11
```

pregunta_1

1. Obtener todas las encuestas junto con sus secciones y preguntas.



findAll surveys
↓
for each survey
↓
find sections by survey_id
↓
for each section
↓
find questions by section_id

El problema es **`@JsonIgnore` en sections`**, porque eso evita que **Spring devuelva las secciones (y por lo tanto las preguntas)** en el JSON.

Entonces aunque el `findAll()` sí trae los datos desde la base de datos, **no se serializan en la respuesta**.

2️⃣ Problema actual en tu código

Aquí:

```java
@OneToMany(mappedBy = "survey")
@ToString.Exclude
@JsonIgnore
private List<Section> sections;
```

`@JsonIgnore` hace que **Spring NO incluya sections en el JSON.**

Por eso tu endpoint devuelve solo:

```json
[
 {
   "id": 1,
   "name": "Encuesta",
   "intro": "...",
   ...
 }
]
```

y **no salen sections ni questions.**

1. Traer todas las surveys
2. Por cada survey:
buscar sus sections
3. Por cada section:
buscar sus questions
4. Hacer set de las listas
5. devolver surveys completas

3️⃣ Implementación del Service (la lógica que te piden)

Java

```java
@Service
public class SurveyServiceImpl implements ISurveyService {

    @Autowired
    private ISurveyRepository surveyRepository;

    @Autowired
    private ISectionRepository sectionRepository;

    @Autowired
    private IQuestionRepository questionRepository;

    @Override
    public List<Survey> findAllWithSectionsAndQuestions() {

        List<Survey> surveys = surveyRepository.findAll();

        for (Survey survey : surveys) {

            // buscar secciones de la encuesta
            List<Section> sections = sectionRepository.findBySurveyId(survey.getId());

            for (Section section : sections) {

                // buscar preguntas de la sección
                List<Question> questions = questionRepository.findBySectionId(section.getId());

                section.setQuestions(questions);
            }

            survey.setSections(sections);
        }

        return surveys;
    }
}
```

```java
@Autowired
private ISurveyService surveyService;

@Autowired
private IInterviewService interviewService;

// ====================================================
// Consulta 1
// Obtener todas las encuestas junto con sus secciones y preguntas
// ====================================================
@GetMapping
public List<Survey> getAll() {
    return surveyService.findAllWithSectionsAndQuestions();
}
```

```java
import java.util.List;

public interface ISurveyService {
    List<Survey> findAllWithSectionsAndQuestions();

    List<Survey> searchByName(String name);

}

```

```java
@Override
public List<Survey> findAllWithSectionsAndQuestions() {

    List<Survey> surveys = surveyRepository.findAll();

    for (Survey survey : surveys) {

        // buscar secciones de la encuesta
        List<Section> sections = sectionRepository.findBySurvey_Id(survey.getId());

        for (Section section : sections) {

            // buscar preguntas de la sección
            List<Question> questions = questionRepository.findBySection_Id(section.getId());

            section.setQuestions(questions);
        }

        survey.setSections(sections);
    }

    return surveys;
}
```

survevy_id

1️⃣ `findBySurveyId`

findBySurveyId(Integer surveyId)

Spring lo interpreta como:

buscar donde Question.surveyId = ?

Es decir, busca un **campo llamado `surveyId` dentro de `Question`**.

Entonces espera algo así en tu entidad:

private Integer surveyId;

o

@Column(name="survey_id")
private Integer surveyId;

Si **no existe esa propiedad**, aparece el error:

No property 'surveyId' found for type 'Question'

2️⃣ `findBySurvey_Id`

findBySurvey_Id(Integer surveyId)

Aquí pasa algo distinto.
El _ le dice a Spring:

"Navega dentro del objeto".

Spring lo interpreta como:

Question.survey.id

o sea:

```
Question
  └── survey (objeto Survey)
        └── id
```

Entonces espera una entidad así:

@ManyToOne

```
@JoinColumn(name = "survey_id")
private Survey survey;
```

y dentro de `Survey`:

```
@Id
private Integer id;
```

Entonces la consulta que genera sería equivalente a:

```
SELECT *
FROM question q
WHERE q.survey_id = ?
```

3️⃣ Ejemplo visual

Supongamos esta entidad:

```
@Entity
public class Question {

  @Id
  private Integer id;

  @ManyToOne
  private Survey survey;
}
```

y

```
@Entity
public class Survey {

  @Id
  private Integer id;
}
```

Método 1

findBySurveyId()

Spring busca:

Question.surveyId

❌ **No existe**

Método 2

findBySurvey_Id()

Spring entiende:

Question.survey.id

✔ **Sí existe**

4️⃣ Regla fácil para recordar

Si en tu entidad tienes:

private Survey survey;

entonces debes usar:

findBySurvey_Id

Si tuvieras:

private Integer surveyId;

entonces usarías:

findBySurveyId

5️⃣ Ejemplos muy comunes en Spring

Entidad	Método	Qué significa
`private Survey survey`	`findBySurvey`	busca por el objeto
`private Survey survey`	`findBySurvey_Id`	busca por id del survey
`private Survey survey`	`findBySurvey_Name`	busca por nombre del survey

`private Integer surveyId`	`findBySurveyId`	busca por ese campo

💡 **Tip importante:**
En **JPA casi siempre se usa la forma con objeto**, por eso en proyectos Spring verás mucho:

findBySurvey_Id
findBySection_Id
findByUser_Id

creation time, you already make sure that the parsed property is a property of the managed domain class. However, you can also define constraints by traversing nested properties. Consider the following method signature:

```java
List<Person> findByAddressZipCode(ZipCode zipCode);
```

Assume a `Person` has an `Address` with a `ZipCode`. In that case, the method creates the `x.address.zipCode` property traversal. The resolution algorithm starts by interpreting the entire part (`AddressZipCode`) as the property and checks the domain class for a property with that name (uncapitalized). If the algorithm succeeds, it uses that property. If not, the algorithm splits up the source at the camel-case parts from the right side into a head and a tail and tries to find the corresponding property — in our example, `AddressZip` and `Code`. If the algorithm finds a property with that head, it takes the tail and continues building the tree down from there, splitting the tail up in the way just described. If the first split does not match, the algorithm moves the split point to the left (`Address`, `ZipCode`) and continues.

Although this should work for most cases, it is possible for the algorithm to select the wrong property. Suppose the `Person` class has an `addressZip` property as well. The algorithm would match in the first split round already, choose the wrong property, and fail (as the type of `addressZip` probably has no `code` property).

To resolve this ambiguity you can use `_` inside your method name to manually define traversal points. So our method name would be as follows:

```java
List<Person> findByAddress_ZipCode(ZipCode zipCode);
```

Because we treat the underscore character as a reserved character, we strongly advise following standard Java naming conventions (that is, not using underscores in property names but using camel case instead).

metodo_primera

1️⃣ Paso 1 — Traer todas las encuestas

```
List<Survey> surveys = surveyRepository.findAll();
```

Esto hace una consulta SQL como:

```
SELECT * FROM survey;
```

Resultado (ejemplo):

Survey 1
Survey 2
Survey 3

En Java ahora tienes una **lista de objetos Survey**:

surveys = [survey1, survey2, survey3]

Pero **estas encuestas todavía NO tienen secciones dentro**.

2️⃣ Paso 2 — Recorrer cada encuesta

```
for (Survey survey : surveys)
```

Esto significa:

tomar survey1
tomar survey2
tomar survey3

Uno por uno.

3️⃣ Paso 3 — Buscar las secciones de esa encuesta

```
List<Section> sections = sectionRepository.findBySurveyId(survey.getId());
```

Esto ejecuta algo como:

```
SELECT * FROM section WHERE survey_id = ?
```

Ejemplo:

Si `survey1.id = 1`

Resultado:

Section A
Section B
Section C

Entonces ahora tienes:

sections = [sectionA, sectionB, sectionC]

4️⃣ Paso 4 — Recorrer cada sección

for (Section section : sections)

Esto toma:

sectionA
sectionB
sectionC

5️⃣ Paso 5 — Buscar preguntas de cada sección

List<Question> questions = questionRepository.findBySectionId(section.getId());

SQL equivalente:

SELECT * FROM question WHERE section_id = ?

Ejemplo:

Question1
Question2
Question3

6️⃣ Aquí aparece el famoso `set`

section.setQuestions(questions);

¿Qué significa `set`?

`set` **asigna un valor a una propiedad del objeto**.

En el objeto `Section` existe algo como:

```
private List<Question> questions;
```

y Lombok (`@Data`) genera automáticamente:

```
public void setQuestions(List<Question> questions) {
  this.questions = questions;
}
```

Entonces esta línea:

```
section.setQuestions(questions);
```

significa:

guardar las preguntas dentro de la sección

Visualmente:

ANTES:

```
Section A
questions = null
```

DESPUÉS:

```
Section A
questions = [Q1, Q2, Q3]
```

---

7 Luego se hace lo mismo con las encuestas

```
survey.setSections(sections);
```

Significa:

guardar las secciones dentro de la encuesta

ANTES:

```
Survey 1
sections = null
```

DESPUÉS:

Survey 1
sections = [sectionA, sectionB, sectionC]

8️⃣ Estructura final construida

Después de todos los `set` queda algo así en memoria:

```
Survey
  Sections
    Questions
```

Más claro:

```
Survey 1
  Section A
     Question 1
     Question 2

  Section B
     Question 3
     Question 4
```

9️⃣ Finalmente se devuelve todo

return surveys;

Spring convierte eso automáticamente a **JSON**.

🔟 Resumen ultra simple

El código hace esto:

1 traer surveys
2 para cada survey buscar sections
3 para cada section buscar questions
4 guardar questions dentro de section (set)
5 guardar sections dentro de survey (set)
6 devolver todo

🎯 Qué significa `set` en una frase

`set` = guardar un valor dentro de un objeto.

Ejemplo simple:

```
Person p = new Person();

p.setName("Luis");
p.setAge(20);
```

Resultado:

```
Person
name = Luis
age = 20
```

razonamiento_db

1️⃣ Sí, en la base de datos ya existe la relación

En tu **base de datos** la relación ya está guardada por las **foreign keys**.

Ejemplo:

Tabla `survey`

id	name
1	Customer Survey

Tabla `section`

id	title	survey_id
1	Personal Info	1
2	Preferences	1

Tabla `question`

id	name	section_id
1	Age	1
2	Gender	1
3	Favorite color	2

Entonces la base de datos **ya sabe** que:

```
Survey 1
  Section 1
    Question 1
    Question 2
  Section 2
    Question 3
```

Pero eso **solo existe en la base de datos**.

2️⃣ Cuando haces `findAll()` NO trae todo automáticamente

Cuando haces:

List<Survey> surveys = surveyRepository.findAll();

Spring ejecuta algo como:

SELECT * FROM survey;

Entonces en memoria tienes:

Survey 1

Pero **NO trae las secciones todavía**.

3️⃣ Por eso hacemos otro `find`

Luego hacemos:

sectionRepository.findBySurveyId(survey.getId());

Eso ejecuta:

SELECT * FROM section WHERE survey_id = 1;

Ahora sí obtenemos:

Section 1
Section 2

Pero todavía **no están dentro del objeto Survey**.

4️⃣ Aquí entra el `set`

Cuando haces:

survey.setSections(sections);

lo que haces es **construir la estructura en memoria**.

Antes:

Survey
id = 1
sections = null

Después:

Survey
id = 1
sections = [Section1, Section2]

No cambia la base de datos.
Solo **organiza los objetos en memoria** para poder devolverlos como JSON.

5️⃣ Lo mismo pasa con las preguntas

Antes del `set`:

Section
questions = null

Después:

Section
questions = [Question1, Question2]

6️⃣ Muy importante: `set` NO guarda en la base de datos

Esto es clave.

section.setQuestions(questions);

❌ NO hace esto:

INSERT
UPDATE

Solo hace esto:

objeto Java en memoria

---

7 La analogía más fácil

Imagina la base de datos como **un archivador**.

📂 Survey
📂 Section
📂 Question

Cada carpeta tiene papeles.

Cuando haces consultas estás **sacando papeles del archivador**.

Pero el `set` es como **ordenarlos en una pila para mostrarlos**.

---

8 Entonces el flujo real es

Base de datos
↓
consultas SQL
↓
objetos Java
↓
set() organiza relaciones
↓
Spring convierte a JSON

---

9 Por qué el profesor quiere que hagas esto

Porque normalmente JPA hace esto automáticamente con **Lazy/Eager loading**.

Pero como te obligaron a usar:

@JsonIgnore

entonces debes **armar la jerarquía manualmente**.

🔟 Regla mental que te servirá siempre

Base de datos

foreign keys

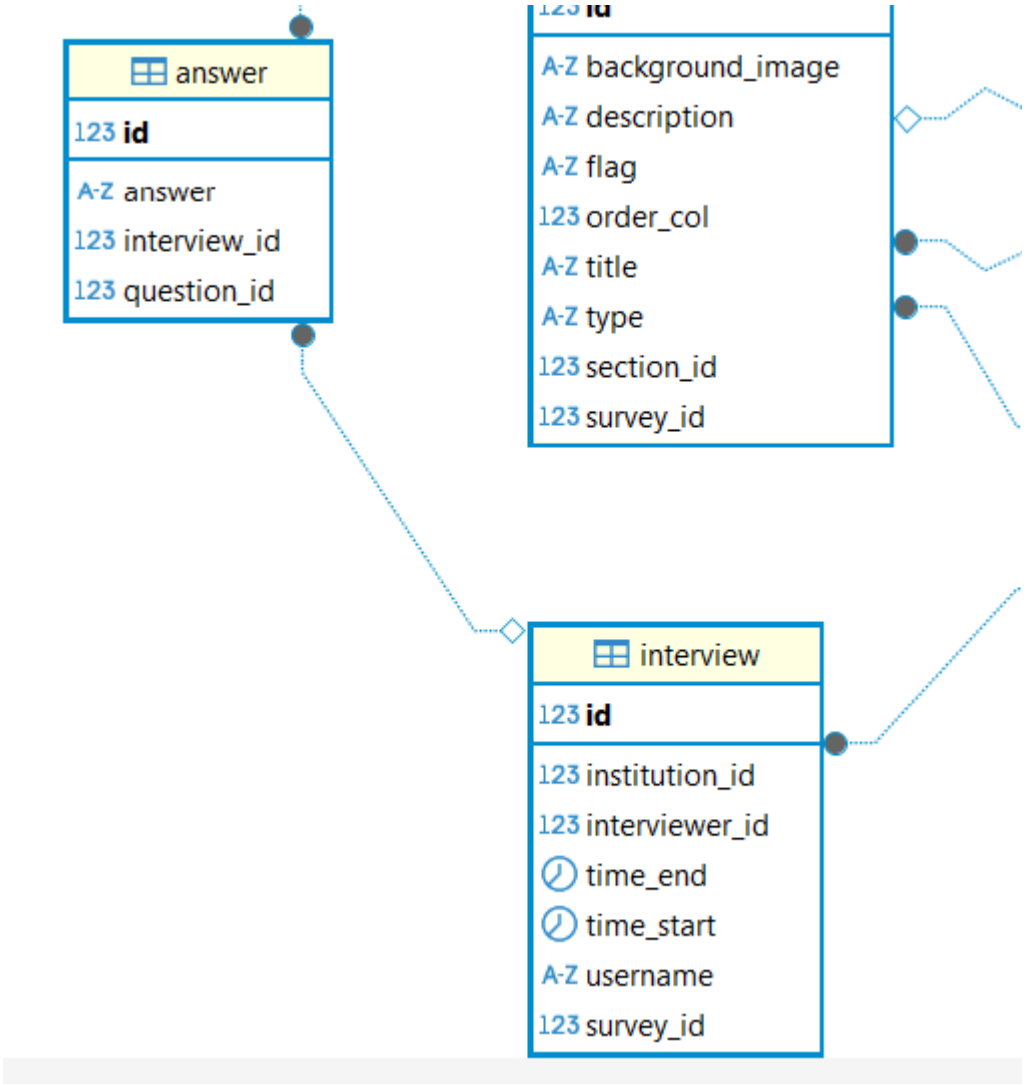
Java

objetos conectados con listas

`set()` sirve para **conectar esos objetos**.

pregunta_2

2. Obtener todas las respuestas de una entrevista específica.



una interview tiene una o muchas respuestas:

```
public class Answer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String answer;

    @ManyToOne
    @ToString.Exclude
    @JsonIgnore
    @JoinColumn(name = "question_id")
    private Question question;

    @ManyToOne
    @ToString.Exclude
    @JoinColumn(name = "interview_id")
    private Interview interview;

}
```

no es necesario en este caso pero:

```java
> main > java > com > example > demo > model > Interview.java > Interview
public class Interview {
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private LocalDateTime timeStart;

    private LocalDateTime timeEnd;

    private Integer interviewerId;

    private String username;

    private Integer institutionId;

    @ManyToOne
    @JoinColumn(name = "survey_id")
    private Survey survey;

    @OneToMany(mappedBy = "interview")
    @ToString.Exclude
    @JsonIgnore
    private List<Answer> answers;

}
```

```java
// ====================================================
// Consulta 2
// Obtener todas las respuestas de una entrevista específica.
//
// Endpoint de ejemplo:
// GET http://localhost:8081/compunet2-2026/answer/interview/1
// ====================================================
@GetMapping("/interview/{id}")
public List<Answer> getAnswersByInterview(@PathVariable Integer id) {
    return answerService.getAnswersByInterview(id);
}
```

Answer.java 5, M | AnswerController.java | IAnswerService.java | ISectionService.java 1

src > main > java > com > example > demo > service > IAnswerService.java > IAnswerService > getAns

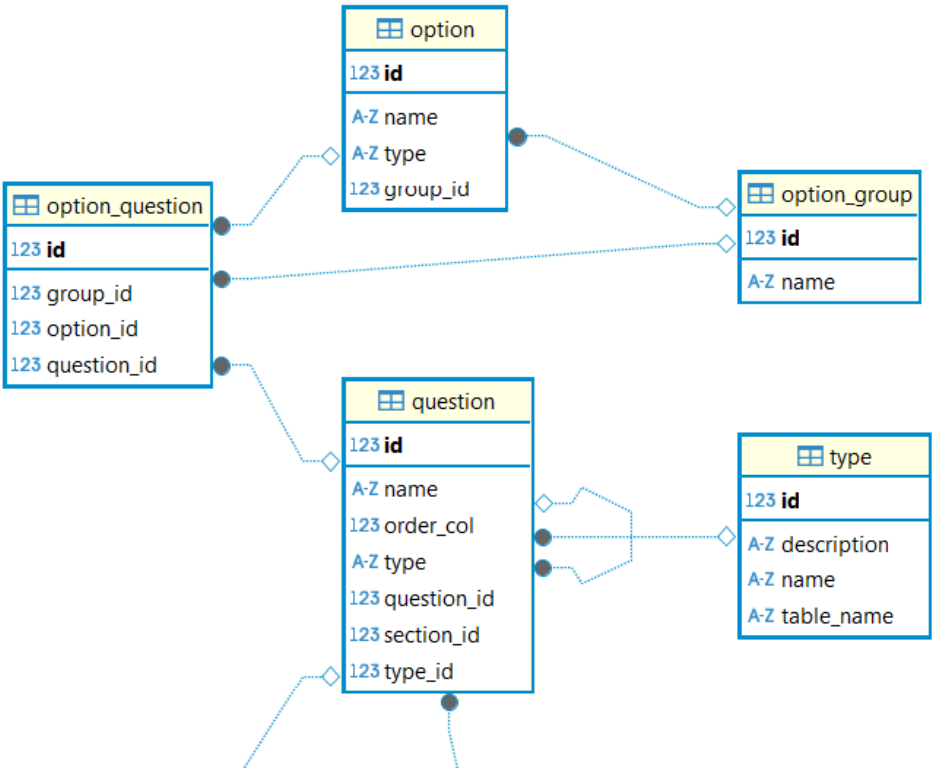
```java
package com.example.demo.service;

import com.example.demo.model.Answer;
import java.util.List;

public interface IAnswerService {

    List<Answer> getAnswersByInterview(Integer interviewId);

    List<Answer> getAnswersByQuestion(Integer questionId);
}
```

```java

@Service
public class AnswerService implements IAnswerService {

    private final IAnswerRepository answerRepository;

    public AnswerService(IAnswerRepository answerRepository) {
        this.answerRepository = answerRepository;
    }

    @Override
    public List<Answer> getAnswersByInterview(Integer interviewId) {
        return answerRepository.findByInterviewId(interviewId);
    }

    @Override
    public List<Answer> getAnswersByQuestion(Integer questionId) {
        return answerRepository.findByQuestion_Id(questionId);
    }
}
```

```java

import org.springframework.stereotype.Repository;
import org.springframework.data.jpa.repository.JpaRepository;
import com.example.demo.model.Answer;
import java.util.List;

@Repository
public interface IAnswerRepository extends JpaRepository<Answer, Integer> {

    List<Answer> findByInterviewId(Integer interviewId);

    List<Answer> findByQuestion_Id(Integer questionId);

}

```

pregunta_3

3. Obtener todas las opciones disponibles para una pregunta específica.



question_id → option_question → option_id

Para **“Obtener todas las opciones disponibles para una pregunta específica”** con tu esquema, las tablas que intervienen son **solo estas tres**:

1. `question` – la pregunta de la que partes (tienes su `id`).
2. `option_question` – la **tabla puente** que relaciona preguntas con opciones.
3. `option` – las opciones que finalmente quieres devolver.

```
question
  id
   |
   | (question_id)
   ▼
option_question
  question_id
  option_id
   |
   | (option_id)
   ▼
option
```

id
name
type

1.lista de OptionQuestion

1 Primer paso: encontrar las relaciones

```
List<OptionQuestion> relations =
    optionQuestionRepository.findByQuestionId(questionId);
```

Aquí se consulta la **tabla puente**:

option_question

Ejemplo de datos:

id	question_id	option_id
1	5	10
2	5	11
3	5	12

Entonces si la pregunta es **5**, obtenemos:

```
relations = [
(question_id=5, option_id=10),
(question_id=5, option_id=11),
(question_id=5, option_id=12)
]
```

Es decir:

Pregunta 5 → opciones 10, 11, 12

2 Crear la lista final

```
List<Option> options = new ArrayList<>();
```

Aquí creamos una lista vacía donde guardaremos las opciones.

Al inicio:

options = [ ]

3 Recorrer cada relación

for (OptionQuestion relation : relations)

Esto significa:

tomar relación 1
tomar relación 2
tomar relación 3

Una por una.

---

4 Buscar la opción real

```
Option option = optionRepository
    .findById(relation.getOption().getId())
    .orElse(null);
```

Aquí usamos el **option_id** de la tabla puente.

Ejemplo:

relation.option.id = 10

Entonces se ejecuta algo como:

SELECT * FROM option WHERE id = 10;

Resultado:

id	name
10	Yes

Con un getOption en repository de OptionQuestion nos traemos todas las option

```java

@Data
@Entity
@Table(name = "option_question")
public class OptionQuestion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @ManyToOne
    @JoinColumn(name = "question_id")
    private Question question;

    @ManyToOne
    @JoinColumn(name = "option_id")
    private Option option;

    @ManyToOne
    @JoinColumn(name = "group_id")
    private OptionGroup group;

}
```

```java

import com.example.demo.model.Section;
import com.example.demo.model.Survey;

import java.util.List;

public interface ISectionService {
    List<Section> getRootSectionsBySurvey(Integer surveyId);

}

```

```java
@Override
public List<Option> getOptionsByQuestion(Integer questionId) {

    List<OptionQuestion> relations = optionQuestionRepository.findByQuestion_Id(questionId);

    List<Option> options = new ArrayList<>();

    for (OptionQuestion relation : relations) {

        options.add(relation.getOption());
    }

    return options;
}
```

```java

@Repository
public interface IOptionQuestionRepository extends JpaRepository<OptionQuestion, Integer> {

    // consulta 11
    List<OptionQuestion> findByQuestion_Id(Integer questionId);

}
```

getOption

1️⃣ De dónde sale `getOption()`

En tu entidad `OptionQuestion` debes tener algo parecido a esto:

```
@Data
@Entity
@Table(name = "option_question")
public class OptionQuestion {

  @Id
  private Integer id;

  @ManyToOne
  @JoinColumn(name = "option_id")
  private Option option;

  @ManyToOne
  @JoinColumn(name = "question_id")
  private Question question;

}
```

Observa este atributo:

```
private Option option;
```

2️⃣ Qué hace Lombok (`@Data`)

Cuando usas:

```
@Data
```

Lombok genera automáticamente:

```
public Option getOption() {
  return option;
}

public void setOption(Option option) {
  this.option = option;
}
```

Entonces **`getOption()` es el getter del atributo `option`**.

3️⃣ Qué significa en la práctica

Cuando haces:

relation.getOption()

estás diciendo:

dame la opción asociada a esta relación

Recuerda que `OptionQuestion` representa una fila de:

option_question

Ejemplo:

id	question_id	option_id
1	5	10

Entonces ese objeto en Java sería algo así:

```
OptionQuestion
  id = 1
  question = Question(5)
  option = Option(10)
```

4️⃣ Entonces esto:

relation.getOption().getId()

significa:

```
relation
  ↓
option
  ↓
id
```

Visualmente:

```
OptionQuestion
   ↓
```

Option
↓
id

Ejemplo real:

relation.getOption().getId()

→ 10

---

5 Por eso luego haces

optionRepository.findById(...)

porque ya tienes el **id de la opción**.

---

6 Resumen simple

Código	Significa
`private Option option`	atributo del objeto
`getOption()`	método que devuelve ese atributo
`relation.getOption()`	obtener la opción de esa relación
`relation.getOption().getId()`	obtener el id de esa opción

Pestaña 16

si la consigna dice no es obligatoria…. o,

PreParcial-1

ordenadas por tiempo: Asc

Utilizo el repositorio de lo que necesito encontrar (encontrar entregas: deliveryRepository, ….es decir obtener todas las posiciones registradas : dronePositionRepository)

[20%] Encontrar las entregas (**Deliveries**) completadas por un dron específico según el **código del dron**.

me salen las 3

```
@GetMapping("/drone/{code}")
public List<Delivery> getDeliveriesByDrone(@PathVariable String code) {
    return deliveryRepository.findByDroneCodeAndStatus(code, status: "COMPLETED");
}
```

[20%] Obtener el recorrido de un dron por su **código**, es decir, obtener todas las posiciones registradas (**DronePositions**) ordenadas por tiempo para el dron seleccionado.

findBy[Atributo][Operador][OrderBy...]

📌 **Traducción mental**

```
Repositorio<X> → empiezas desde X
```

🧭 **3. Cómo construir consultas SIN equivocarte**

Usa este patrón:

```
findBy + ruta_de_atributos + condiciones + orden
```

```
@GetMapping("/route/{code}")
public List<DronePosition> getDroneRoute(@PathVariable String code) {
    return dronePositionRepository.findByDrone_CodeOrderByRecordedAtAsc(code);
}
```

[20%] Obtener el **último evento registrado** (**DroneEvent**) para una entrega específica según el **id de la entrega**.

findBy + RUTA_DE_ATRIBUTOS + OrderBy + ATRIBUTO + Asc/Desc

```
@GetMapping("/droneEvent/{delivery_id}")
public DroneEvent getLatestEvent(@PathVariable Long deliveryId) {
    return droneEventRepository.findFirstByDelivery_IdOrderByEventTimeDesc(deliveryId);
}
```

```
public interface IDroneEventRepository extends JpaRepository<DroneEvent, Long> {
    /*

    Optional<DroneEvent> findTopByDeliveryIdOrderByEventTimeDesc(Long deliveryId);
     */

    DroneEvent findFirstByDeliveryIdOrderByEventTimeDesc(Long deliveryId);

}
```

- [20%] Encontrar los buses que atienden una ruta específica por nombre de la ruta

analisis:
Con busRepository, necesitamos una lista de buses y seria tipo: findByRouteName

[20%] Encontrar los buses que atienden una ruta específica por nombre de la ruta

```
@GetMapping("/route/{routeName}")
public List<Bus> getBusesByRouteName(@PathVariable String routeName) {
    return busRepository.findByRouteRouteName(routeName);
}
```

```
public interface IBusRepository extends JpaRepository<Bus, Integer> {

    //Encontrar los buses que atienden una ruta específica por nombre de la ruta
    List<Bus> findByRouteRouteName(String routeName);

}
```

- [20%] Obtener el recorrido de un bus por placa, esto es, obtener cada posición registrada ordenada por tiempo para el bus seleccionado

obtener cada posición: GeoPoint, esto quiere decir que repository es geopositionRepository, luego

```
@GetMapping("/bus/{licensePlate}")
public List<GeoPoint> getGeoPointsByLicensePlate(@PathVariable String licensePlate) {
    return geoPointsRepository.findByBusLicensePlateOrderByTimestampAsc(licensePlate);
}
```

- [20%] Obtener la ubicación más reciente registrada de un bus según su matrícula

obtener la ubicacion: location (GeoPoint)

otra vez geopointController luego seria

```java
@GetMapping("/bus/{licensePlate}/latest")
public GeoPoint getLatestGeoPointByLicensePlate(@PathVariable String licensePlate) {
    return geoPointsRepository.findFirstByBusLicensePlateOrderByTimestampDesc(licensePlate);
}
```

🔥 La diferencia REAL: `Asc` vs `Desc`

Todo depende de **qué quieres obtener**.

🧠 Caso 1: Recorrido del bus (todas las posiciones)

> "obtener cada posición registrada ordenada por tiempo"

Aquí quieres **TODO el historial**, en orden cronológico:

```java
findByBusLicensePlateOrderByTimestampAsc(String licensePlate)
```

✔️ ¿Por qué `Asc`?

Porque quieres:

```
08:00 → 08:05 → 08:10 → 08:15 → ...
```

➡️ Es decir, **del más antiguo al más reciente**

👉 Esto tiene sentido porque estás viendo el **recorrido completo**

🧠 Caso 2: Última ubicación

> "obtener la ubicación más reciente"

Aquí NO quieres todo, solo **una**:

```java
findFirstByBusLicensePlateOrderByTimestampDesc(String licensePlate)
```

[20%] Encontrar los buses que atienden rutas por un tipo particular. Por ejemplo, obtener los buses que atienden rutas tipo Alimentador

encontrar los buses : busRepository.

```
@GetMapping("/type/{type}")
public List<Bus> getBusesByRouteType(@PathVariable String type) {
    return busRepository.findByRouteType(type);
}
```

JoinColumn

El name de JoinColumn debe coincidir con el de la DB.

🧠 RESPUESTA CORTA

👉 **Sí depende de la BD... PERO también del modelo**

No es solo uno. Es una relación entre ambos.

⚖️ LA VERDAD COMPLETA

1. La BD define la columna física

Ejemplo en tu `data.sql`:

```sql
drone_id
delivery_id
```

👉 Eso es lo que debes poner en:

```java
@JoinColumn(name = "drone_id")
```

🔄 CÓMO SE CONECTAN

```java
@ManyToOne
@JoinColumn(name = "drone_id")
private Drone drone;
```

💥 Traducción:

```
ATRIBUTO JAVA: drone
↓
COLUMNA BD: drone_id
↓
RELACIÓN: apunta a Drone.id
```

🚨 IMPORTANTE (lo que te confundía)

👉 Aunque en Java NO tengas un campo llamado `drone_id`...

✔️ JPA lo usa internamente porque tú lo dijiste en `@JoinColumn`

Entonces, si una entidad tiene un ManyToOne (JoinColumn) entonces ese name estara en la bd:

✅ RESPUESTA CORTA

👉 **SÍ:**

El `name` de `@JoinColumn` es **una columna REAL en la BD**

```java
@JoinColumn(name = "drone_id")
```

✔ significa que en la tabla existe:

```sql
drone_id
```

```sql
INSERT INTO deliveries (id, tracking_code, status, destination, assigned_at, drone_id) VALUES
    (1, 'DEL-2026-0001', 'COMPLETED', 'Calle 10 # 20-30', '2026-03-13 08:00:00', 1),
    (2, 'DEL-2026-0002', 'IN_PROGRESS', 'Avenida 6N # 14-22', '2026-03-13 09:10:00', 2),
    (3, 'DEL-2026-0003', 'COMPLETED', 'Cra 80 # 5-11', '2026-03-13 10:15:00', 1),
    (4, 'DEL-2026-0004', 'PENDING', 'Calle 45 # 30-18', '2026-03-13 11:20:00', 2),
    (5, 'DEL-2026-0005', 'COMPLETED', 'Carrera 56 # 8-40', '2026-03-13 12:05:00', 4),
    (6, 'DEL-2026-0006', 'IN_PROGRESS', 'Calle 70 # 12-90', '2026-03-13 12:40:00', 4),
    (7, 'DEL-2026-0007', 'PENDING', 'Avenida 3 # 22-15', '2026-03-13 13:00:00', 5),
    (8, 'DEL-2026-0008', 'COMPLETED', 'Calle 16 # 101-09', '2026-03-13 13:25:00', 1);
```

no puede faltar y además de eso, debe coincidir.

ojo con esto tambien:

💥 ¿Qué pasó?

En tu `data.sql` tienes esto:

```sql
INSERT INTO bus (id, license_plate, route_id, type)
VALUES (1, 'AAA111', 1, "TIPO1");
```

El problema es:

👉 **Estás usando comillas dobles `" "` en vez de comillas simples `' '`**

🧠 Regla clave en SQL

- `'texto'` → ✅ **String válido**
- `"texto"` → ❌ **Se interpreta como nombre de columna**

Entonces H2 entiende esto como:

```sql
"TIPO1"  → una columna llamada TIPO1
```

Y como esa columna no existe → 💣 error

---

loomback

en pom.xml

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.32</version>
    <scope>provided</scope>
</dependency>
```

luego:
mvn clean install

db_

si en el data.sql esta en singular, el Entity debe ir en singula (debe llamarse ingual)r:

Bus.java | IBusRepository.java | data.sql | GeoPointController.java | BusController.java | GeoPoint.java

ExamenJPATemplateA-main > ExamenJPATemplateA-main > src > main > resources > data.sql > "TIPO6"

```sql
-- Routes
INSERT INTO route (id, route_name, type) VALUES (1, 'E21', 'Expreso');
INSERT INTO route (id, route_name, type) VALUES (2, 'E31', 'Expreso');
INSERT INTO route (id, route_name, type) VALUES (3, 'T31', 'Troncal');
INSERT INTO route (id, route_name, type) VALUES (4, 'A16', 'Alimentador');

-- Buses
INSERT INTO bus (id, license_plate, route_id, type) VALUES (1, 'AAA111', 1, "TIPO1");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (2, 'BBB222', 1, "TIPO1");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (3, 'CCC333', 2, "TIPO2");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (4, 'DDD444', 2, "TIPO2");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (5, 'EEE555', 3, "TIPO3");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (6, 'FFF666', 3, "TIPO3");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (7, 'GGG777', 4, "TIPO4");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (8, 'HHH888', 4, "TIPO4");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (9, 'III999', 1, "TIPO5");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (10, 'JJJ000', 2, "TIPO5");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (11, 'KKK111', 3, "TIPO6");
INSERT INTO bus (id, license_plate, route_id, type) VALUES (12, 'LLL222', 4, "TIPO6");

-- GeoPoints
-- Bus 1
INSERT INTO geo_point (id, latitude, longitude, timestamp, bus_id) VALUES (1, 3.4516, -76.5320, '2025-09-12 10:00
INSERT INTO geo_point (id, latitude, longitude, timestamp, bus_id) VALUES (2, 3.4521, -76.5330, '2025-09-12 10:10
```

```java
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@Entity(name = "route")
public class Route {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String routeName;

    private String type;

    @OneToMany(mappedBy = "route", cascade = CascadeType.ALL)
    private List<Bus> buses;
```

en este ejemplo se hace en plural:

re-parcial-jpa-Giuseppe ff > src > main > resources > data.sql > ...

```sql
INSERT INTO drones (id, code, name, status) VALUES
(1, 'DRN-001', 'Falcon One', 'AVAILABLE'),
(2, 'DRN-002', 'Eagle Wing', 'BUSY'),
(3, 'DRN-003', 'Sky Runner', 'MAINTENANCE'),
(4, 'DRN-004', 'Hawk Eye', 'BUSY'),
(5, 'DRN-005', 'Nimbus Jet', 'AVAILABLE');




INSERT INTO deliveries (id, tracking_code, status, destination, assigned_at, drone_id) VALUES
    (1, 'DEL-2026-0001', 'COMPLETED', 'Calle 10 # 20-30', '2026-03-13 08:00:00', 1),
    (2, 'DEL-2026-0002', 'COMPLETED', 'Avenida 6N # 14-22', '2026-03-13 09:10:00', 1),
    (3, 'DEL-2026-0003', 'COMPLETED', 'Cra 80 # 5-11', '2026-03-13 10:15:00', 1),
    (4, 'DEL-2026-0004', 'PENDING', 'Calle 45 # 30-18', '2026-03-13 11:20:00', 2),
    (5, 'DEL-2026-0005', 'COMPLETED', 'Carrera 56 # 8-40', '2026-03-13 12:05:00', 4),
    (6, 'DEL-2026-0006', 'IN_PROGRESS', 'Calle 70 # 12-90', '2026-03-13 12:40:00', 4),
    (7, 'DEL-2026-0007', 'PENDING', 'Avenida 3 # 22-15', '2026-03-13 13:00:00', 5),
    (8, 'DEL-2026-0008', 'COMPLETED', 'Calle 16 # 101-09', '2026-03-13 13:25:00', 1);


INSERT INTO drone_events (id, event_type, event_time, description, delivery_id) VALUES
    (1, 'MISSION STARTED', '2026-03-13 08:01:00', 'Mission started for delivery 1', 1),
```

```java
@Data
@NoArgsConstructor
@Entity(name = "drones")

public class Drone {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String code;

    private String name;

    private String status;

    // • Un Drone puede estar asociado a múltiples Deliveries.

    @OneToMany(mappedBy = "drone")
    private List<Delivery> deliveries;

    // • Un Drone puede registrar múltiples DronePositions a lo largo de su
    // operación
    @OneToMany(mappedBy = "drone")
    private List<DronePosition> dronePositions;

}
```

Or_In

IN:

Variantes de examen

- “...varios tipos”
- “...una lista de tipos”
- “...los siguientes tipos”
- “...un conjunto de tipos”
- “...tipos dados”

clave: **más de 2 / lista**

enunciado:

Obtener los buses que atienden rutas de tipo **Alimentador o Troncal**.

✅ Opción directa (la más típica)

repository:

```
List<Bus> findByRouteTypeOrRouteType(String type1, String type2);
```

✔ Controller

```
@GetMapping("/type")
public List<Bus> getBusesByMultipleTypes(
      @RequestParam String type1,
      @RequestParam String type2) {
   return busRepository.findByRouteTypeOrRouteType(type1, type2);
}
```

🚀 URL

http://localhost:8085/pre-parcial/buses/type?type1=Alimentador&type2=Troncal

—----------

OR:

enunciado:

Obtener los buses que atienden rutas de los siguientes tipos: Alimentador, Troncal.

Mejor forma (más elegante)

Si pueden ser muchos tipos (no solo 2), usa **IN**:

✔ Repository

```
List<Bus> findByRouteTypeIn(List<String> types);
```

✔ Controller

```
@GetMapping("/types")
public List<Bus> getBusesByTypes(@RequestParam List<String> types) {
  return busRepository.findByRouteTypeIn(types);
}
```

🚀 URL

http://localhost:8085/pre-parcial/buses/types?types=Alimentador,Troncal

http://localhost:8085/pre-parcial/buses/types/Alimentador,Troncal

---

🧠 Diferencia clave

Método	Cuándo usar
`Or`	pocos valores (2 típicamente)
`In`	muchos valores (más flexible)

—----------

Ejemplo de OR

encontrar deliveries por destination o status

http://localhost:8085/pre-parcial/delivery/search?status=COMPLETED
http://localhost:8085/pre-parcial/delivery/search?destination=Cali
http://localhost:8085/pre-parcial/delivery/search?status=COMPLETED&destination=Cali

```
       @GetMapping("/search")
public List<Delivery> searchDeliveries(
        @RequestParam(required = false) String status,
        @RequestParam(required = false) String destination) {

    if (status != null && destination != null) {
        return
deliveryRepository.findByDroneStatusOrDestination(status, destination);
    } else if (status != null) {
        return deliveryRepository.findByStatus(status);
    } else if (destination != null) {
        return deliveryRepository.findByDestination(destination);
    } else {
        return deliveryRepository.findAll();
    }
}
}
```

```
package com.example.demo.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.demo.model.Delivery;

public interface IDeliveryRepository extends JpaRepository<Delivery,
Long> {

    List<Delivery> findByDroneCodeAndStatus(String droneCode, String
status);
```

```
    List<Delivery> findByDroneStatusOrDestination(String status, String
destination);
    List<Delivery> findByStatus(String status);
    List<Delivery> findByDestination(String destination);
}
```

DISTINCT

“sin repetir” / “únicos” → Distinct

1. DISTINCT básico (sobre entidades)

Enunciado

Obtener los **buses distintos** que atienden rutas de tipo específico.

Repository

```
List<Bus> findDistinctByRouteType(String type);
```

```
@GetMapping("/type/distinct/{type}")
public List<Bus> getDistinctBusesByRouteType(@PathVariable String type) {
return busRepository.findDistinctByRouteType(type);
}
```

http://localhost:8085/pre-parcial/buses/type/Alimentador

---

2. DISTINCT + OR

Enunciado

Obtener los **buses sin repetir** que atienden rutas de tipo A o tipo B.

Repository

```
List<Bus> findDistinctByRouteTypeOrRouteType(String type1, String type2);
```

version con pathvariable:

```
@GetMapping("/type/distinct/{type1}/{type2}")
public List<Bus> getDistinctBusesByTwoTypes(
@PathVariable String type1,
@PathVariable String type2) {
return busRepository.findDistinctByRouteTypeOrRouteType(type1, type2);
}
```

http://localhost:8085/pre-parcial/buses/type/distinct/Alimentador/Troncal

---

3. DISTINCT + IN (muy común)

Enunciado

Obtener los **buses sin duplicados** que atienden rutas de varios tipos.

Repository

```
List<Bus> findDistinctByRouteTypeIn(List<String> types);
```

http://localhost:8085/pre-parcial/buses/types?types=Alimentador,Troncal

```
@GetMapping("/types/distinct")
public List<Bus> getDistinctBusesByTypes(@RequestParam List<String> types) {
return busRepository.findDistinctByRouteTypeIn(types);
}
```

---

4. DISTINCT sobre atributos (clave de examen)

Enunciado

Obtener los **tipos de rutas sin repetir**.

Repository

```
List<String> findDistinctRouteTypeBy();
```

Aquí ya **no devuelves Bus**, sino el atributo (`String`)

http://localhost:8085/pre-parcial/buses/types/distinct

```
@GetMapping("/route-types")

public List<String> getDistinctRouteTypes() {

    return busRepository.findDistinctRouteTypeBy();
```

```
}
```

---

5. DISTINCT + ORDER BY

Enunciado

> Obtener los **buses distintos** que atienden rutas de tipo específico, **ordenados por placa**.

Repository

```
List<Bus> findDistinctByRouteTypeOrderByLicensePlateAsc(String type);
```

http://localhost:8085/pre-parcial/buses/type/Alimentador/sorted

```
@GetMapping("/type/{type}/ordered")
public List<Bus> getDistinctOrdered(@PathVariable String type) {
    return busRepository.findDistinctByRouteTypeOrderByLicensePlateAsc(type);
}
```

---

6. DISTINCT + IN + ORDER (nivel pro 😈)

Enunciado

> Obtener los **buses sin repetir** que atienden rutas de varios tipos, **ordenados por placa**.

Repository

```
List<Bus> findDistinctByRouteTypeInOrderByLicensePlateAsc(List<String> types);
```

http://localhost:8085/pre-parcial/buses/types/sorted?types=Alimentador,Troncal

```
@GetMapping("/types/ordered")
public List<Bus> getDistinctOrderedByTypes(@RequestParam List<String> types) {
    return busRepository.findDistinctByRouteTypeInOrderByLicensePlateAsc(types);
}
```

---

7. DISTINCT en relaciones (muy evaluado)

Enunciado

Obtener las **rutas distintas** que tienen buses asociados.

Repository (desde Bus)

```
List<Route> findDistinctRouteBy();
```

http://localhost:8085/pre-parcial/buses/routes/distinct

```
@GetMapping("/routes/distinct")
public List<Route> getDistinctRoutes() {
    return busRepository.findDistinctRouteBy();
}
```

RESUMEN PARA EXAMEN

Si ves en el enunciado:

- “sin repetir” / “únicos” → **Distinct**
- “varios tipos” → **In**
- “o” → **Or**
- “ordenados” → **OrderBy**

Tip mental rápido

findDistinctByRouteTypeInOrderByLicensePlateAsc

Se lee como:

👉 “Buscar buses distintos donde el tipo de ruta esté en una lista, ordenados por placa”

AND

¿Qué significa AND?

En un enunciado:

> "...que cumplan **dos condiciones al mismo tiempo**"

Ejemplo:

- tipo de ruta **Y** tipo de bus
- tipo de ruta **Y** placa
- tipo de ruta **Y** fecha

Ejemplo base (muy típico)

Enunciado

> Obtener los buses que atienden rutas de tipo **Alimentador Y** que sean de tipo **TIPO1**

Repository

```
List<Bus> findByRouteTypeAndType(String routeType, String busType);
```

Controller

```
@GetMapping("/filter/{routeType}/{busType}")
public List<Bus> getBusesByRouteTypeAndBusType(
      @PathVariable String routeType,
      @PathVariable String busType) {
   return busRepository.findByRouteTypeAndType(routeType, busType);
}
```

URL

http://localhost:8085/pre-parcial/buses/filter/Alimentador/TIPO1

Traducción a SQL

```
WHERE route.type = 'Alimentador'
AND bus.type = 'TIPO1'
```

IgnoreCase

findByAssociationsMemberFullNameIgnoreCaseAndAssociationsTerminationDateIsNull(name)

[20%] Obtener las asociaciones activas de un miembro dado su nombre, es decir, aquellas donde no se ha registrado una fecha de terminación.

`IsNull` ⇒ **no ha terminado.**

GreaterThan

¿Qué significa `GreaterThan`?

`GreaterThan` es una **palabra clave (keyword)** que Spring Data JPA reconoce automáticamente.

👉 Traducción directa:

PerformanceMetricGreaterThan

significa:

performance_metric > ?

🔍 Cómo lo interpreta JPA

Tu método:

findByLocationAndHostGroupParticipantsPerformanceMetricGreaterThan(location, metric)

Spring lo descompone así:

Parte	Significado
`findBy`	tipo de consulta
`Location`	filtro directo
`HostGroupParticipantsPerformanceMetric`	navegación entre entidades
`GreaterThan`	operador `>`

⚡ Ejemplo sencillo para que lo veas claro

Si tuvieras una entidad `Person`:

findByAgeGreaterThan(18)

👉 JPA genera algo equivalente a:

SELECT * FROM person WHERE age > 18;

🔥 Otras keywords similares (muy importantes)

Spring Data JPA tiene varias:

Método	Significa
`GreaterThan`	`>`
`LessThan`	`<`
`GreaterThanEqual`	`>=`
`LessThanEqual`	`<=`
`Between`	`BETWEEN`
`IsNull`	`IS NULL`
`IsNotNull`	`IS NOT NULL`
`Like`	`LIKE`

🎯 En tu caso específico

findByLocationAndHostGroupParticipantsPerformanceMetricGreaterThan(location, metric)

👉 Significa:

> **“Eventos en una ubicación dada donde el grupo anfitrión tiene al menos un participante cuyo valor de desempeño es mayor que el valor recibido.”**

⚠️ Ojo con este detalle (muy importante)

Tu interpretación fue:

> “participantes cuyo valor de desempeño es mayor que el valor recibido”

🔥 Otras keywords similares (muy importantes)

Spring Data JPA tiene varias:

Método	Significa
`GreaterThan`	`>`
`LessThan`	`<`
`GreaterThanEqual`	`>=`
`LessThanEqual`	`<=`
`Between`	`BETWEEN`
`IsNull`	`IS NULL`
`IsNotNull`	`IS NOT NULL`
`Like`	`LIKE`

ignoreCase

En **Spring Data JPA**, ignorar mayúsculas/minúsculas se hace con palabras clave dentro del nombre del método. Aquí tienes **todas las formas útiles** 👇

🔹 1. `IgnoreCase` (la más común)

Se aplica a **un solo campo**:

findByNameIgnoreCase(String name)

👉 Traducción:

WHERE LOWER(name) = LOWER(?)

🔹 2. `ContainingIgnoreCase` (búsqueda parcial)

findByNameContainingIgnoreCase(String name)

👉 Traducción:

WHERE LOWER(name) LIKE LOWER('%valor%')

✔️ Sirve para búsquedas tipo “contiene” (muy usado en filtros)

🔹 3. `StartsWithIgnoreCase`

findByNameStartsWithIgnoreCase(String name)

👉 Traducción:

WHERE LOWER(name) LIKE LOWER('valor%')

🔹 4. `EndsWithIgnoreCase`

findByNameEndsWithIgnoreCase(String name)

👉 Traducción:

WHERE LOWER(name) LIKE LOWER('%valor')

🔹 5. `AllIgnoreCase` (varios campos)

Cuando tienes varios atributos:

findByFirstNameAndLastNameAllIgnoreCase(String first, String last)

👉 Aplica ignore case a **todos los campos**

🔹 6. Combinado con relaciones (como tu caso)

findByAssociationsMemberFullNameIgnoreCase(String name)

✔️ Funciona igual aunque haya navegación entre entidades

⚠️ Importante (esto cae en parciales)

- ❌ No es automático → debes escribir `IgnoreCase`
- ❌ Solo funciona con **String**
- ✔️ JPA lo convierte internamente usando `LOWER()` (o equivalente)

🎯 Resumen rápido

Método	Qué hace
`IgnoreCase`	Igual sin importar mayúsculas
`ContainingIgnoreCase`	Contiene (LIKE %...%)
`StartsWithIgnoreCase`	Empieza por
`EndsWithIgnoreCase`	Termina en
`AllIgnoreCase`	Aplica a varios campos

// 3. Eventos en una ubicación específica donde la región de origen tiene al menos un miembro con rating > valor

```
// http://localhost:8081/partial2jap/events-by-location-and-rating?location=ZoneA&rating=85
@GetMapping("/events-by-location-and-rating")
public List<Event> getEventsByLocationAndRating(@RequestParam String location,
                                                @RequestParam Integer rating) {
    return eventRepo.findByLocationAndOriginRegionMembersRatingGreaterThan(location,
rating);
}
```

Pestaña 22

🧠 1. La “documentación” real que necesitas

No es tanto leer todo, sino entender el patrón:

👉 **Spring Data JPA Query Methods**

Búscalo así:

Spring Data JPA derived query methods

Pero lo importante no es leer todo, sino entender esta idea:

find + By + Campo + Operador + Campo + Operador...

🔥 2. Ejemplos clave (esto es lo que debes dominar)

✅ DISTINCT

✔ Repository

```
List<Bus> findDistinctByRouteType(String type);
```

🧠 Qué hace

```
SELECT DISTINCT *
FROM bus b
JOIN route r ON ...
WHERE r.type = ?
```

👉 Evita duplicados

🔥 Otro ejemplo útil

```
List<String> findDistinctRouteTypeBy();
```

👉 Esto devuelve **solo los tipos de ruta sin repetir**

🚀 3. Operadores que MÁS salen en parciales

Te dejo los importantes:

📌 BÁSICOS

findByLicensePlate(String plate)
findByType(String type)

📌 RELACIONES (lo más importante)

findByRouteRouteName(String name)
findByRouteType(String type)

📌 OR / AND

findByRouteTypeOrRouteType(String t1, String t2)
findByTypeAndLicensePlate(String type, String plate)

📌 IN

findByRouteTypeIn(List<String> types)

📌 ORDER BY

findByRouteTypeOrderByLicensePlateAsc(String type)
findByRouteTypeOrderByLicensePlateDesc(String type)

📌 TOP / FIRST (muy importante)

findFirstByBusLicensePlateOrderByTimestampDesc(String plate)

👉 último registro

📌 DISTINCT

findDistinctByRouteType(String type)

📌 LIKE (búsquedas)

findByLicensePlateContaining(String text)

📌 BETWEEN

findByTimestampBetween(Timestamp start, Timestamp end)

🧠 4. Regla de oro (esto te cambia todo)

Spring lee esto como un **path**:

```
findByRouteType
    ↓
bus.route.type
```

🔥 5. Cómo practicar (lo que te recomiendo)

Hazte ejercicios así:

👉 Traduce esto a método:

- “buses de rutas tipo Alimentador”
- “buses con placa que contenga AAA”
- “última ubicación de un bus”
- “buses de varias rutas”
- “buses ordenados por placa”

💣 6. Ejemplo tipo examen (con TODO)

List<Bus> findDistinctByRouteTypeInOrderByLicensePlateAsc(List<String> types);

👉 Esto hace:

- DISTINCT
- IN
- ORDER BY

📚 7. Si quieres documentación real

Te dejo lo clave:

👉 **Spring Data JPA Reference**
Sección:

- *Query Creation*
- *Repository query keywords*

🧠 8. Consejo real (esto te sube nivel rápido)

No memorices…

👉 **interpreta el nombre como si fuera SQL**

Ejemplo:

findByRouteTypeInOrderByLicensePlateAsc

Piensa:

WHERE route.type IN (...)
ORDER BY license_plate ASC